

# J5

## Intégration et projets

Assembler toute la semaine en une chaîne exécutable d'une seule commande — puis défendre son projet en soutenance.

10-integration

projet-final

# Objectifs de la journée



## La chaîne complète

lecture → validation → traitement → restitution : le modèle fourni.



## Une seule commande

argparse : options propres, aide automatique, échec explicite.



## Finir le projet

Matinée : dernier jalon. La chaîne tourne de bout en bout.



## Soutenir

10 minutes de démonstration, 5 de questions — par groupe.

### LA SEMAINE

J1

Fondamentaux du langage

J2

Fonctions & structures

J3

Fichiers & robustesse

J4

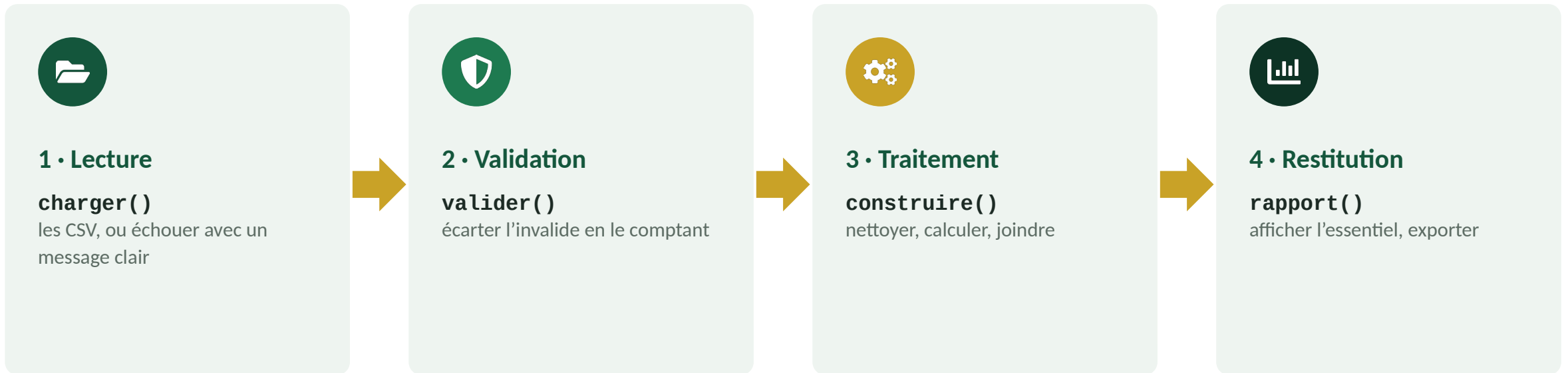
Objets & pandas

J5

Intégration & projets

*Cet après-midi : les soutenances.*

# Quatre étapes, toujours les mêmes



**10-integration/pipeline.py** est entièrement rédigé et commenté : c'est le MODÈLE du projet. Lisez-le, exécutez-le, imitez-le.



```
uv run python pipeline.py --region CKY
```

```
| uv run python pipeline.py --toutes
```

# argparse : le programme devient un outil



## Des options nommées

--region CKY, --toutes : lisible six mois plus tard.



## --help gratuit

argparse documente l'outil tout seul.



## Groupe mutuellement exclusif

--region OU --toutes, pas les deux : le module refuse.



## main() raconte l'histoire

Quatre appels, quatre print numérotés — on suit l'exécution.



pipeline.py

```
parser = argparse.ArgumentParser()
g = parser.add_mutually_exclusive_group(
    required=True)
g.add_argument("--region")
g.add_argument("--toutes",
    action="store_true")

def main():
    print("1. Chargement...")
    tables = charger()
    ...
```

# Échouer proprement, tracer ce qu'on écarte



## Jamais de traceback brut

`sys.exit("message + remède")` : l'utilisateur sait quoi faire.



## Chaque filtre annonce son bilan

« CA invalides écartés : 26 » — la donnée perdue est visible.



## C'est un critère de la grille

Robustesse : 3 points sur 20. Vérifié en soutenance.



échec explicite

```
try:
    contr = pd.read_csv(...)
except FileNotFoundError as exc:
    sys.exit(
        f"Manquant : {exc.filename}\n"
        f"→ python generate_data.py")
```

```
$ python pipeline.py --toutes
```

```
1. Chargement...
```

```
CA invalides écartés      : 26
```

```
Dates inexploitable      : 0
```

# Le socle : vous n'écrivez que votre sujet

```
le squelette

projet-final/socle/projet.py

def charger():      # FOURNI
def valider():      # À VOUS DE JOUER
def traiter():      # À VOUS DE JOUER
def restituer():    # À VOUS DE JOUER
def main():         # FOURNI
```

*Exécuté tel quel, le socle s'arrête sur `NotImplementedError` : c'est votre point de départ, pas un bug.*



## La structure est imposée

Quatre fonctions, un main : comme pipeline.py.



## Remplir dans l'ordre

valider → traiter → restituer → options argparse.



## Interdits

versionner les données produites ; le copier-coller non compris.



## README de groupe

quoi, comment lancer, qui a fait quoi.

# La grille d'évaluation — 20 points

| Critère                                                   | Points |
|-----------------------------------------------------------|--------|
| La chaîne s'exécute de bout en bout sans erreur           | 6      |
| Justesse : le résultat répond à la question du sujet      | 5      |
| Robustesse : données sales gérées, messages clairs        | 3      |
| Lisibilité : fonctions courtes, noms parlants, docstrings | 3      |
| Soutenance : clarté, démonstration, réponses              | 3      |



BONUS +1

un graphique matplotlib  
pertinent dans le rapport

6 points sur 20 tiennent en une phrase : ÇA TOURNE. Sécurisez l'exécution avant d'embellir.

# La soutenance : 10 + 5 minutes



**2 min**

**Le sujet** — La question métier, avec vos mots — pas de jargon.



**5 min**

**La démonstration** — UNE commande, en direct. Le bilan des lignes écartées commenté.



**3 min**

**Un choix technique** — Une difficulté rencontrée et comment vous l'avez tranchée.



**5 min**

**Questions** — Chacun répond sur SA partie — d'où le « qui a fait quoi » du README.

**Conseil :** testez la démonstration sur un poste PROPRE (clone frais + uv sync + génération des données) — c'est là que ça casse.

# Continuer à progresser



## Le dépôt reste

Solutions complètes, aide-mémoire (docs/), pipeline modèle : tout reste accessible et clonable.



## Automatiser un vrai besoin

Choisissez UNE tâche répétitive de votre service et scriptez-la : c'est ainsi qu'on ancre.



## Approfondir

[docs.python.org](https://docs.python.org) en français, [pandas.pydata.org](https://pandas.pydata.org) — et les modules 06/09 à relire dans un mois.

*Prochaine marche naturelle : tests avec pytest, environnements de production, bases de données (SQL) — le paquet fiscalite est prêt à grandir.*

# À retenir

- ✓ **Un environnement reproductible** — uv sync : la compétence invisible qui évite 80 % des « ça marche pas chez moi ».
- ✓ **Le réflexe de la donnée sale** — valider, compter ce qu'on écarte, échouer proprement.
- ✓ **Du script au paquet** — fiscalite/ : votre code est devenu réutilisable.
- ✓ **pandas** — charger, nettoyer, joindre, agréger — le quotidien de l'analyse.
- ✓ **Un projet soutenu** — une chaîne complète, exécutée en une commande, défendue en public.



**Félicitations — et merci !**

Le dépôt reste ouvert : git pull pour les dernières solutions. Vos retours nourriront la prochaine session.